Onion Disease & Pest Classification — OnionGuard ML Pipeline

Assigned to: Nathaniel Adika
Dataset: TOM2024 (Onion diseases + pests)
Platform: Kaggle Notebook (GPU)
Model: MobileNetV3-Large (ImageNet pretrained)
Target: >90% accuracy, <4MB TFLite quantized model
Classes: 6 (Alternaria, Bulb Blight, Caterpillar, Fusarium, Healthy, Virosis)

Pipeline Overview

1. **EDA** — Class Distribution, Image Stats, Pixel Intensity, Sample Grids
2. **Data Preparation** — Train/Val/Test splits, class weights for imbalance
3. **Model Training** — 3-stage progressive fine-tuning (AdamW + Cosine LR)
4. **Evaluation** — Confusion matrix, classification report, misclassified samples
5. **Grad-CAM** — Interpretability / XAI visualizations
6. **TFLite Export** — INT8 quantization for mobile deployment

Techniques Used

Component	Technique
Model	MobileNetV3-Large (ImageNet pretrained)
Input	224×224
Optimizer	AdamW (lr=1e-4, weight_decay=0.01)
LR Schedule	Cosine annealing + linear warmup
Augmentation	RandomFlip/Rotation/Zoom/Brightness/Contrast
Regularization	Dropout(0.3) + Class weights
Fine-tuning	3-stage progressive unfreezing
Quantization	INT8 Post-Training Quantization
XAI	Grad-CAM on last conv layer

> **Checkpointing**: Every major step saves results to `/kaggle/working/` so you can download them from the Output tab after the run.

In [145…

```
!rm -rf /kaggle/working/combined_dataset /kaggle/working/caterpillar_output/checkpo
print("Cleaned up combined_dataset and checkpoints. Fresh start.")
```

Cleaned up combined_dataset and checkpoints. Fresh start.

In [146…

```
# ── 0.1 Install deps & setup paths (Kaggle) ──────────────────────────────
import os, pickle

!pip install -q opencv-python-headless

# ── Kaggle dataset path (added via "+ Add Data" in notebook sidebar) ──────
KAGGLE_DATASET = "/kaggle/input/datasets/souravparija3025/tom2024-dataset/TOM2024_d

if not os.path.exists(KAGGLE_DATASET):
    raise FileNotFoundError(
        f"{KAGGLE_DATASET} not found. "
        "Make sure you added the tom2024-dataset via + Add Data in the right panel.
    )
print(f"Dataset found at: {KAGGLE_DATASET}")

import random
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image
from pathlib import Path
from collections import Counter

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, callbacks, optimizers
from tensorflow.keras.applications import MobileNetV3Large
from sklearn.metrics import classification_report, confusion_matrix
import cv2

random.seed(42)
np.random.seed(42)
tf.random.set_seed(42)

sns.set_style('whitegrid')
%matplotlib inline

# ── BASE path (Kaggle — no extraction needed, dataset is pre-mounted) ──────
BASE = Path(KAGGLE_DATASET)
print(f"BASE: {BASE}")

# ── Outputs saved to /kaggle/working/ (downloadable from Output tab) ──────
OUT_DIR  = Path("/kaggle/working/caterpillar_output")
CKPT_DIR = OUT_DIR / "checkpoints"
OUT_DIR.mkdir(parents=True, exist_ok=True)
CKPT_DIR.mkdir(parents=True, exist_ok=True)

# ── Checkpoint helpers ────────────────────────────────────────────────────
def save_checkpoint(name, data):
    with open(CKPT_DIR / f"{name}.pkl", "wb") as f:
        pickle.dump(data, f)
    print(f"  [CHECKPOINT SAVED] {name}")
```

```
def load_checkpoint(name):
    path = CKPT_DIR / f"{name}.pkl"
    if path.exists():
        with open(path, "rb") as f:
            data = pickle.load(f)
        print(f"  [CHECKPOINT LOADED] {name}")
        return data
    return None

print(f"TensorFlow: {tf.__version__}")
print(f"GPU available: {len(tf.config.list_physical_devices('GPU')) > 0}")
if tf.config.list_physical_devices('GPU'):
    print(f"GPU: {tf.config.list_physical_devices('GPU')[0]}")
print(f"Output dir : {OUT_DIR}")
print(f"Checkpoints: {CKPT_DIR}")
print("All imports OK")
```

```
Dataset found at: /kaggle/input/datasets/souravparija3025/tom2024-dataset/TOM2024_da
taset
BASE: /kaggle/input/datasets/souravparija3025/tom2024-dataset/TOM2024_dataset
TensorFlow: 2.19.0
GPU available: False
Output dir : /kaggle/working/caterpillar_output
Checkpoints: /kaggle/working/caterpillar_output/checkpoints
All imports OK
```

Part 1 — EDA & Class Distribution Audit

In [147…

```
# ── Configuration ──────────────────────────────────────────────────────
IMG_SIZE = 224
BATCH_SIZE = 32
NUM_CLASSES = 6
CLASS_NAMES = [
    "Alternaria",
    "Bulb_Blight",
    "Caterpillar",
    "Fusarium",
    "Healthy",
    "Virosis",
]
EPOCHS_PHASE1 = 20
EPOCHS_PHASE2 = 25
EPOCHS_PHASE3 = 15

# ── Dataset paths (all onion classes) ──────────────────────────────────
RAW_PATHS = {
    "Alternaria":   BASE / "onion_diseases" / "Alternaria_D",
    "Bulb_Blight":  BASE / "onion_diseases" / "Bulb_blight-D",
    "Caterpillar":  BASE / "onion_pests" / "Caterpillar-P",
    "Fusarium":     BASE / "onion_diseases" / "Fusarium-D",
    "Healthy":      BASE / "onion_diseases" / "Healthy_leaf",
    "Virosis":      BASE / "onion_diseases" / "Virosis-D",
}
```

```python
# These are not used for multi-class (no augmented folder), but keep for EDA compat
AUG_TRAIN = RAW_PATHS.copy()
AUG_TEST = RAW_PATHS.copy()

IMG_EXTS = {".jpg", ".jpeg", ".png", ".bmp", ".tiff", ".webp"}

def list_images(folder: Path) -> list:
    if not folder.exists():
        print(f"  WARNING: {folder} does not exist!")
        return []
    return [f for f in folder.iterdir() if f.suffix.lower() in IMG_EXTS]

# Verify paths and count images
print("Path verification:")
total_images = 0
class_counts = {}
for name, p in RAW_PATHS.items():
    count = len(list_images(p)) if p.exists() else 0
    class_counts[name] = count
    total_images += count
    status = "OK" if p.exists() else "MISSING"
    print(f"  [{status}] {name}: {p} ({count} images)")
print(f"\nTotal: {total_images} images across {NUM_CLASSES} classes")

# ── Compute class weights to handle imbalance ──────────────────────
from sklearn.utils.class_weight import compute_class_weight
total = sum(class_counts.values())
n_classes = len(class_counts)
class_weight_dict = {}
for i, name in enumerate(CLASS_NAMES):
    count = class_counts[name]
    weight = total / (n_classes * count)
    # Cap weights to max 5.0 to prevent tiny classes from destabilizing training
    class_weight_dict[i] = min(weight, 5.0)
print(f"\nClass weights (to handle imbalance):")
for i, name in enumerate(CLASS_NAMES):
    print(f"  {name}: {class_weight_dict[i]:.2f} ({class_counts[name]} images)")
```

```
Path verification:
  [OK] Alternaria: /kaggle/input/datasets/souravparija3025/tom2024-dataset/TOM2024_dataset/onion_diseases/Alternaria_D (515 images)
  [OK] Bulb_Blight: /kaggle/input/datasets/souravparija3025/tom2024-dataset/TOM2024_dataset/onion_diseases/Bulb_blight-D (30 images)
  [OK] Caterpillar: /kaggle/input/datasets/souravparija3025/tom2024-dataset/TOM2024_dataset/onion_pests/Caterpillar-P (879 images)
  [OK] Fusarium: /kaggle/input/datasets/souravparija3025/tom2024-dataset/TOM2024_dataset/onion_diseases/Fusarium-D (738 images)
  [OK] Healthy: /kaggle/input/datasets/souravparija3025/tom2024-dataset/TOM2024_dataset/onion_diseases/Healthy_leaf (679 images)
  [OK] Virosis: /kaggle/input/datasets/souravparija3025/tom2024-dataset/TOM2024_dataset/onion_diseases/Virosis-D (203 images)

Total: 3044 images across 6 classes

Class weights (to handle imbalance):
  Alternaria: 0.99 (515 images)
  Bulb_Blight: 5.00 (30 images)
  Caterpillar: 0.58 (879 images)
  Fusarium: 0.69 (738 images)
  Healthy: 0.75 (679 images)
  Virosis: 2.50 (203 images)
```

2. Image Counts & Class Distribution

In [148…

```
rows = []
for label in CLASS_NAMES:
    raw = list_images(RAW_PATHS[label])
    rows.append({
        "Class": label,
        "Images": len(raw),
    })

df_counts = pd.DataFrame(rows)
df_counts
```

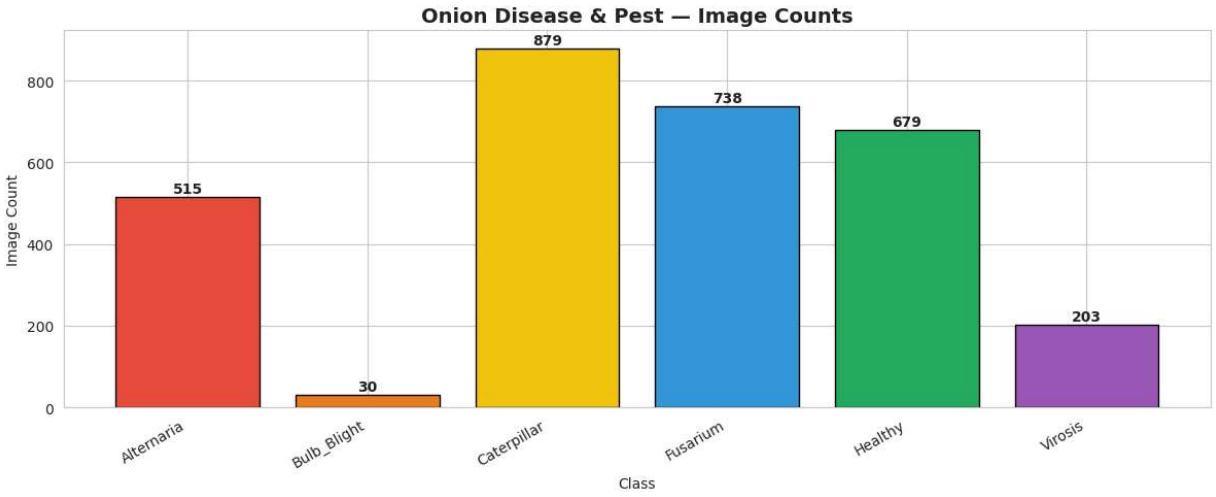
Out[148…

	Class	Images
0	Alternaria	515
1	Bulb_Blight	30
2	Caterpillar	879
3	Fusarium	738
4	Healthy	679
5	Virosis	203

In [149…

```
fig, ax = plt.subplots(figsize=(12, 5))

colors_map = ["#e74c3c", "#e67e22", "#f1c40f", "#3498db", "#27ae60", "#9b59b6"]
bars = ax.bar(df_counts["Class"], df_counts["Images"],
```

```
              color=colors_map, edgecolor="black")
ax.set_title("Onion Disease & Pest — Image Counts", fontsize=14, fontweight="bold")
ax.set_ylabel("Image Count")
ax.set_xlabel("Class")
plt.xticks(rotation=30, ha="right")

for bar, v in zip(bars, df_counts["Images"]):
    ax.text(bar.get_x() + bar.get_width()/2, v + 10, str(v),
            ha="center", fontweight="bold")

plt.tight_layout()
fig.savefig(OUT_DIR / "01_class_distribution.png", dpi=150, bbox_inches="tight")
plt.show()
save_checkpoint("eda_class_dist", df_counts.to_dict())
```




```
[CHECKPOINT SAVED] eda_class_dist
```

3. Image Dimensions & File Size Analysis

In [150…

```
def get_image_stats(img_paths: list, sample_n: int = 300) -> dict:
    """Compute width, height, file size, and channel stats from a sample."""
    sample = random.sample(img_paths, min(sample_n, len(img_paths)))
    widths, heights, file_sizes, channels = [], [], [], []
    corrupt_count = 0

    for p in sample:
        try:
            file_sizes.append(os.path.getsize(p) / 1024)  # KB
            with Image.open(p) as img:
                w, h = img.size
                widths.append(w)
                heights.append(h)
                channels.append(len(img.getbands()))
        except Exception:
            corrupt_count += 1

    return {
        "widths": np.array(widths),
        "heights": np.array(heights),
        "file_sizes_kb": np.array(file_sizes),
```

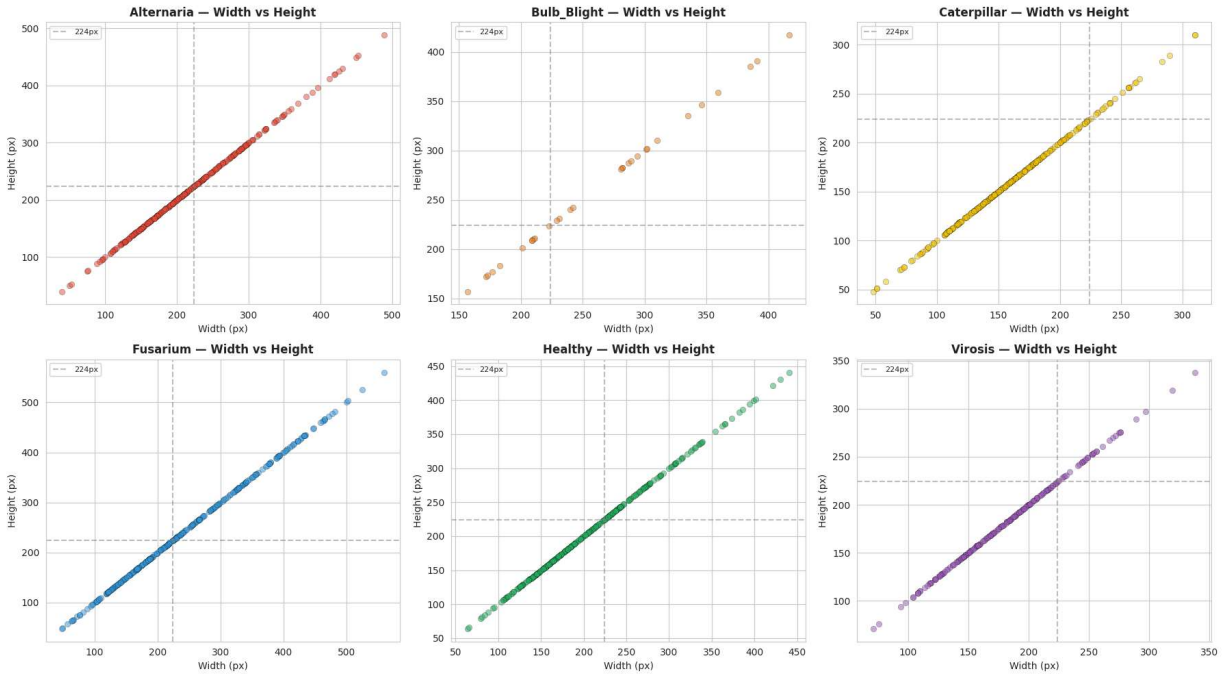
```
        "channels": Counter(channels),
        "corrupt": corrupt_count,
        "sampled": len(sample),
    }

all_stats = {}
for label, path in RAW_PATHS.items():
    imgs = list_images(path)
    stats = get_image_stats(imgs)
    all_stats[label] = stats

    print(f"\n--- {label} (sampled {stats['sampled']} images) ---")
    print(f"  Width  — min: {stats['widths'].min()}, max: {stats['widths'].max()},
          f"mean: {stats['widths'].mean():.0f}, median: {np.median(stats['widths'])
    print(f"  Height — min: {stats['heights'].min()}, max: {stats['heights'].max()}
          f"mean: {stats['heights'].mean():.0f}, median: {np.median(stats['heights'
    print(f"  File size (KB) — min: {stats['file_sizes_kb'].min():.1f}, "
          f"max: {stats['file_sizes_kb'].max():.1f}, mean: {stats['file_sizes_kb'].
    print(f"  Channels: {dict(stats['channels'])}")
    if stats['corrupt'] > 0:
        print(f"  WARNING — Corrupt/unreadable: {stats['corrupt']}")
```

```
--- Alternaria (sampled 300 images) ---
  Width  — min: 40, max: 488, mean: 216, median: 208
  Height — min: 40, max: 488, mean: 216, median: 208
  File size (KB) — min: 1.1, max: 54.1, mean: 11.3
  Channels: {3: 300}

--- Bulb_Blight (sampled 30 images) ---
  Width  — min: 157, max: 417, mean: 264, median: 262
  Height — min: 157, max: 417, mean: 264, median: 262
  File size (KB) — min: 5.8, max: 31.4, mean: 15.6
  Channels: {3: 30}

--- Caterpillar (sampled 300 images) ---
  Width  — min: 48, max: 310, mean: 158, median: 154
  Height — min: 48, max: 310, mean: 158, median: 154
  File size (KB) — min: 1.2, max: 18.5, mean: 6.1
  Channels: {3: 300}

--- Fusarium (sampled 300 images) ---
  Width  — min: 48, max: 560, mean: 246, median: 228
  Height — min: 48, max: 560, mean: 246, median: 228
  File size (KB) — min: 1.1, max: 69.7, mean: 18.7
  Channels: {3: 300}

--- Healthy (sampled 300 images) ---
  Width  — min: 64, max: 441, mean: 206, median: 194
  Height — min: 64, max: 441, mean: 206, median: 194
  File size (KB) — min: 1.8, max: 34.2, mean: 9.9
  Channels: {3: 300}

--- Virosis (sampled 203 images) ---
  Width  — min: 71, max: 338, mean: 181, median: 179
  Height — min: 71, max: 338, mean: 181, median: 179
  File size (KB) — min: 2.1, max: 19.2, mean: 8.6
  Channels: {3: 203}
```

In [151…

```python
# Dimension scatter plot
colors = {name: c for name, c in zip(CLASS_NAMES,
          ["#e74c3c", "#e67e22", "#f1c40f", "#3498db", "#27ae60", "#9b59b6"])}

n_classes = len(all_stats)
cols = min(3, n_classes)
rows = (n_classes + cols - 1) // cols
fig, axes = plt.subplots(rows, cols, figsize=(6 * cols, 5 * rows))
axes = np.array(axes).flatten()

for idx, (label, stats) in enumerate(all_stats.items()):
    axes[idx].scatter(stats["widths"], stats["heights"],
                      alpha=0.5, color=colors.get(label, "gray"), edgecolors="black
    axes[idx].set_title(f"{label} — Width vs Height", fontweight="bold")
    axes[idx].set_xlabel("Width (px)")
    axes[idx].set_ylabel("Height (px)")
    axes[idx].axhline(y=224, color="gray", linestyle="--", alpha=0.5, label="224px"
    axes[idx].axvline(x=224, color="gray", linestyle="--", alpha=0.5)
    axes[idx].legend(fontsize=8)
```

```
# Hide unused axes
for j in range(idx + 1, len(axes)):
    axes[j].set_visible(False)

plt.tight_layout()
fig.savefig(OUT_DIR / "02_dimension_scatter.png", dpi=150)
plt.show()
```



In [152…

```
# File size distribution
fig, ax = plt.subplots(figsize=(12, 5))
for label, stats in all_stats.items():
    ax.hist(stats["file_sizes_kb"], bins=30, alpha=0.5,
            label=label, color=colors.get(label, "gray"), edgecolor="black")
ax.set_xlabel("File Size (KB)")
ax.set_ylabel("Frequency")
ax.set_title("File Size Distribution (Raw Images)", fontweight="bold")
ax.legend(fontsize=9)
plt.tight_layout()
fig.savefig(OUT_DIR / "03_filesize_distribution.png", dpi=150)
plt.show()
```

4. Sample Image Grid

In [153…

```
for label, path in RAW_PATHS.items():
    imgs = list_images(path)
    n_show = min(12, len(imgs))
    if n_show == 0:
        print(f"Skipping {label} — no images")
        continue
    sample = random.sample(imgs, n_show)

    n_cols = min(6, n_show)
    n_rows = (n_show + n_cols - 1) // n_cols
    fig, axes = plt.subplots(n_rows, n_cols, figsize=(3 * n_cols, 3 * n_rows))
    if n_rows == 1 and n_cols == 1:
        axes = np.array([axes])
    axes = np.array(axes).flatten()
    fig.suptitle(f"{label} — Sample Images ({len(imgs)} total)",
                 fontsize=14, fontweight="bold")

    for idx, ax in enumerate(axes):
        if idx < len(sample):
            img = Image.open(sample[idx]).convert("RGB")
            ax.imshow(np.array(img))
            ax.set_title(f"{img.size[0]}x{img.size[1]}", fontsize=8)
        ax.axis("off")

    plt.tight_layout()
    fig.savefig(OUT_DIR / f"04_samples_{label}.png", dpi=150)
    plt.show()
```

Alternaria — Sample Images (515 total)

Bulb_Blight — Sample Images (30 total)

Caterpillar — Sample Images (879 total)

Fusarium — Sample Images (738 total)

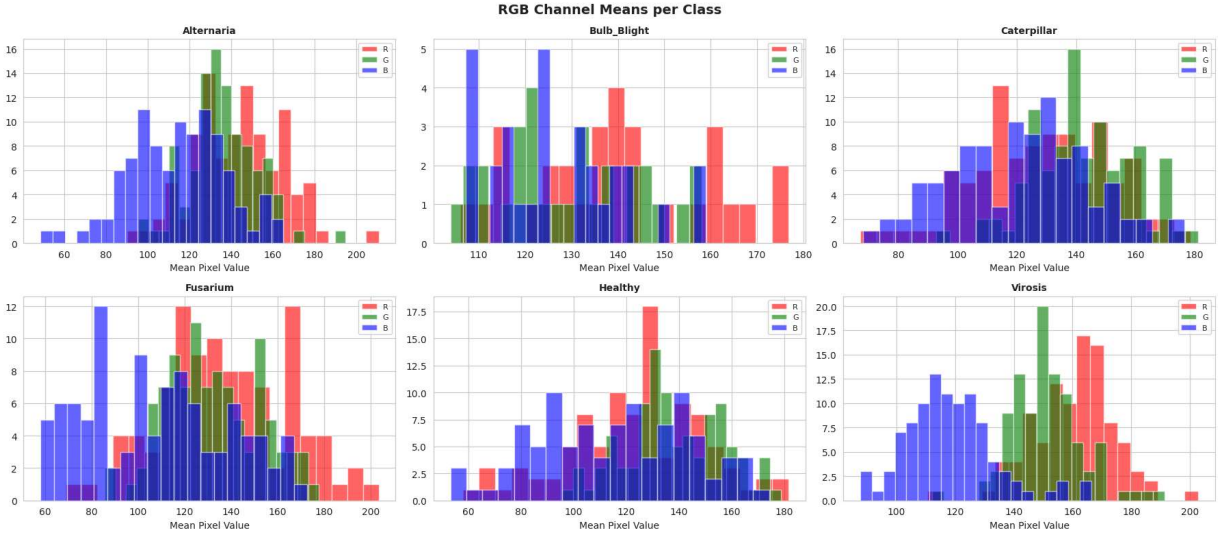
5. Pixel Intensity (RGB Channel Means)

In [154…

```
n_classes = len(RAW_PATHS)
cols = min(3, n_classes)
rows = (n_classes + cols - 1) // cols
fig, axes = plt.subplots(rows, cols, figsize=(6 * cols, 4 * rows))
axes = np.array(axes).flatten()

for idx, (label, path) in enumerate(RAW_PATHS.items()):
    imgs = list_images(path)
    sample = random.sample(imgs, min(100, len(imgs)))
    r_means, g_means, b_means = [], [], []

    for p in sample:
        try:
            img = np.array(Image.open(p).convert("RGB"))
            r_means.append(img[:, :, 0].mean())
            g_means.append(img[:, :, 1].mean())
            b_means.append(img[:, :, 2].mean())
        except Exception:
            pass

    axes[idx].hist(r_means, bins=20, alpha=0.6, color="red", label="R")
    axes[idx].hist(g_means, bins=20, alpha=0.6, color="green", label="G")
    axes[idx].hist(b_means, bins=20, alpha=0.6, color="blue", label="B")
    axes[idx].set_title(f"{label}", fontweight="bold", fontsize=10)
    axes[idx].set_xlabel("Mean Pixel Value")
    axes[idx].legend(fontsize=8)
```

```
    print(f"--- {label} ---")
    print(f"  R: mean={np.mean(r_means):.1f}, G: mean={np.mean(g_means):.1f}, B: me

for j in range(idx + 1, len(axes)):
    axes[j].set_visible(False)

fig.suptitle("RGB Channel Means per Class", fontsize=14, fontweight="bold")
plt.tight_layout()
fig.savefig(OUT_DIR / "05_pixel_intensity.png", dpi=150)
plt.show()
```

```
--- Alternaria ---
  R: mean=143.6, G: mean=135.8, B: mean=114.1
--- Bulb_Blight ---
  R: mean=139.9, G: mean=129.5, B: mean=126.8
--- Caterpillar ---
  R: mean=128.2, G: mean=142.5, B: mean=120.9
--- Fusarium ---
  R: mean=137.6, G: mean=132.5, B: mean=108.2
--- Healthy ---
  R: mean=124.2, G: mean=138.4, B: mean=115.9
--- Virosis ---
  R: mean=160.4, G: mean=151.1, B: mean=118.3
```



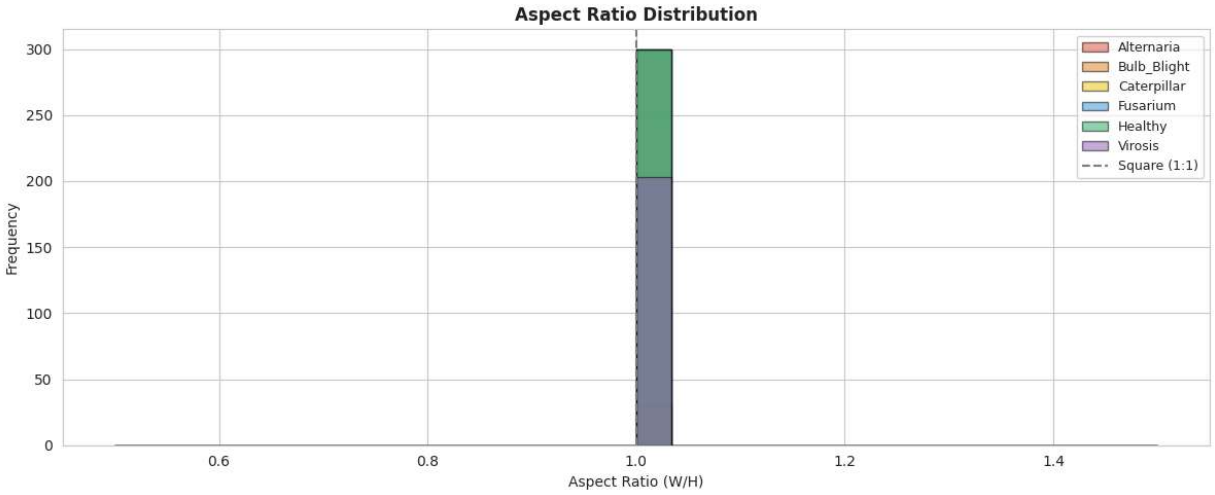
6. Aspect Ratio Distribution

In [155…

```
fig, ax = plt.subplots(figsize=(12, 5))
for label, stats in all_stats.items():
    ratios = stats["widths"] / stats["heights"]
    ax.hist(ratios, bins=30, alpha=0.5, label=label,
            color=colors.get(label, "gray"), edgecolor="black")
    print(f"  {label}: mean ratio={ratios.mean():.2f}, std={ratios.std():.2f}")

ax.set_xlabel("Aspect Ratio (W/H)")
ax.set_ylabel("Frequency")
ax.set_title("Aspect Ratio Distribution", fontweight="bold")
ax.axvline(x=1.0, color="gray", linestyle="--", label="Square (1:1)")
ax.legend(fontsize=9)
plt.tight_layout()
```

```
fig.savefig(OUT_DIR / "06_aspect_ratio.png", dpi=150)
plt.show()
```

```
Alternaria: mean ratio=1.00, std=0.00
Bulb_Blight: mean ratio=1.00, std=0.00
Caterpillar: mean ratio=1.00, std=0.00
Fusarium: mean ratio=1.00, std=0.00
Healthy: mean ratio=1.00, std=0.00
Virosis: mean ratio=1.00, std=0.00
```



7. Augmented vs Raw — Comparison

In [156…

```
# Show sample images from each class
for label in CLASS_NAMES:
    raw_imgs = list_images(RAW_PATHS[label])
    if len(raw_imgs) == 0:
        print(f"Skipping {label} — no images")
        continue

    n_show = min(6, len(raw_imgs))
    fig, axes = plt.subplots(1, n_show, figsize=(3 * n_show, 3))
    if n_show == 1:
        axes = [axes]
    fig.suptitle(f"{label} — Sample Images ({len(raw_imgs)} total)",
                 fontsize=12, fontweight="bold")

    sample = random.sample(raw_imgs, n_show)
    for i, ax in enumerate(axes):
        img = Image.open(sample[i]).convert("RGB")
        ax.imshow(np.array(img))
        ax.set_title(f"{img.size[0]}x{img.size[1]}", fontsize=8)
        ax.axis("off")

    plt.tight_layout()
    fig.savefig(OUT_DIR / f"07_samples_{label}.png", dpi=150)
    plt.show()
```

1.8 EDA Summary

In [157…

```
print("=" * 60)
print("EDA SUMMARY")
print("=" * 60)
print(f"\nDataset: TOM2024 — Onion Disease & Pest Classification")
print(f"Classes: {NUM_CLASSES}")
print()
for i, row in df_counts.iterrows():
    print(f"  {row['Class']:20s}: {row['Images']:4d} images")
print(f"\n  {'TOTAL':20s}: {df_counts['Images'].sum():4d} images")
print(f"\nSmallest class: {df_counts.loc[df_counts['Images'].idxmin(), 'Class']} ({
print(f"Largest class:  {df_counts.loc[df_counts['Images'].idxmax(), 'Class']} ({df
print(f"Imbalance ratio: {df_counts['Images'].max() / df_counts['Images'].min():.1f
print("\nClass weights will be used during training to handle imbalance.")
```

```
save_checkpoint("eda_counts", df_counts.to_dict())
```

```
============================================================
EDA SUMMARY
============================================================

Dataset: TOM2024 — Onion Disease & Pest Classification
Classes: 6

  Alternaria          :  515 images
  Bulb_Blight         :   30 images
  Caterpillar         :  879 images
  Fusarium            :  738 images
  Healthy             :  679 images
  Virosis             :  203 images

  TOTAL               : 3044 images

Smallest class: Bulb_Blight (30 images)
Largest class:  Caterpillar (879 images)
Imbalance ratio: 29.3x

Class weights will be used during training to handle imbalance.
  [CHECKPOINT SAVED] eda_counts
```

Part 2 — Data Preparation

Load Category B augmented data, split train into train/val (85/15), build tf.data pipelines with CutMix and Mixup augmentation.

In [158…

```
# ── 2.1 Build tf.data pipelines from all onion classes ──────────────
import shutil

COMBINED_DIR = Path("/kaggle/working/combined_dataset")

# Remove old combined dataset to start fresh
if COMBINED_DIR.exists():
    shutil.rmtree(COMBINED_DIR)

for class_name, src_path in RAW_PATHS.items():
    dst = COMBINED_DIR / class_name
    dst.mkdir(parents=True, exist_ok=True)
    imgs = list_images(src_path)
    for img_file in imgs:
        os.symlink(str(img_file), str(dst / img_file.name))
    print(f"  {class_name}: {len(imgs)} images")

print(f"Created combined dataset at {COMBINED_DIR}")

# 80/20 split: training vs held-out
train_ds = tf.keras.utils.image_dataset_from_directory(
    COMBINED_DIR,
```

```
    labels="inferred",
    label_mode="categorical",
    class_names=CLASS_NAMES,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    shuffle=True,
    seed=42,
    validation_split=0.2,
    subset="training",
)

held_out_ds = tf.keras.utils.image_dataset_from_directory(
    COMBINED_DIR,
    labels="inferred",
    label_mode="categorical",
    class_names=CLASS_NAMES,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    shuffle=True,
    seed=42,
    validation_split=0.2,
    subset="validation",
)

# Unbatch and shuffle to ensure all classes in val and test
held_out_unbatched = held_out_ds.unbatch().shuffle(2000, seed=42)
total_held = sum(1 for _ in held_out_unbatched)
val_count = total_held // 2
print(f"Total held-out images: {total_held}, Val: {val_count}, Test: {total_held -

# Re-shuffle with same seed for reproducibility
held_out_unbatched = held_out_ds.unbatch().shuffle(2000, seed=42)
val_ds = held_out_unbatched.take(val_count).batch(BATCH_SIZE)
test_ds = held_out_unbatched.skip(val_count).batch(BATCH_SIZE)

train_batches = tf.data.experimental.cardinality(train_ds).numpy()
val_batches = sum(1 for _ in val_ds)
test_batches = sum(1 for _ in test_ds)

print(f"\nTrain batches: {train_batches}")
print(f"Val batches:   {val_batches}")
print(f"Test batches:  {test_batches}")

# Verify all classes in each split
for name, ds in [("Train", train_ds), ("Val", val_ds), ("Test", test_ds)]:
    all_labels = []
    for _, lbls in ds:
        all_labels.extend(np.argmax(lbls.numpy(), axis=1))
    unique, counts = np.unique(all_labels, return_counts=True)
    print(f"\n{name} set:")
    for u, c in zip(unique, counts):
        print(f"  {CLASS_NAMES[u]}: {c}")

save_checkpoint("data_splits", {
    "train_batches": int(train_batches),
    "val_batches":   int(val_batches),
```

```
    "test_batches":  int(test_batches),
})
```

```
  Alternaria: 515 images
  Bulb_Blight: 30 images
  Caterpillar: 879 images
  Fusarium: 738 images
  Healthy: 679 images
  Virosis: 203 images
Created combined dataset at /kaggle/working/combined_dataset
Found 3044 files belonging to 6 classes.
Using 2436 files for training.
Found 3044 files belonging to 6 classes.
Using 608 files for validation.
Total held-out images: 608, Val: 304, Test: 304

Train batches: 77
Val batches:   10
Test batches:  10

Train set:
  Alternaria: 428
  Bulb_Blight: 26
  Caterpillar: 694
  Fusarium: 592
  Healthy: 547
  Virosis: 149

Val set:
  Alternaria: 47
  Caterpillar: 92
  Fusarium: 63
  Healthy: 67
  Virosis: 35

Test set:
  Alternaria: 37
  Bulb_Blight: 4
  Caterpillar: 94
  Fusarium: 73
  Healthy: 65
  Virosis: 31
  [CHECKPOINT SAVED] data_splits
```

In [159…

```
# ── 2.2 Augmentation using tf.image (works reliably on any pixel range) ─────

def augment_image(image):
    """Apply augmentations to a single image."""
    image = tf.image.random_flip_left_right(image)
    image = tf.image.random_brightness(image, max_delta=0.2)
    image = tf.image.random_contrast(image, lower=0.8, upper=1.2)
    image = tf.image.random_saturation(image, lower=0.8, upper=1.2)
    return image

print("Augmentation functions defined.")
```

Augmentation functions defined.

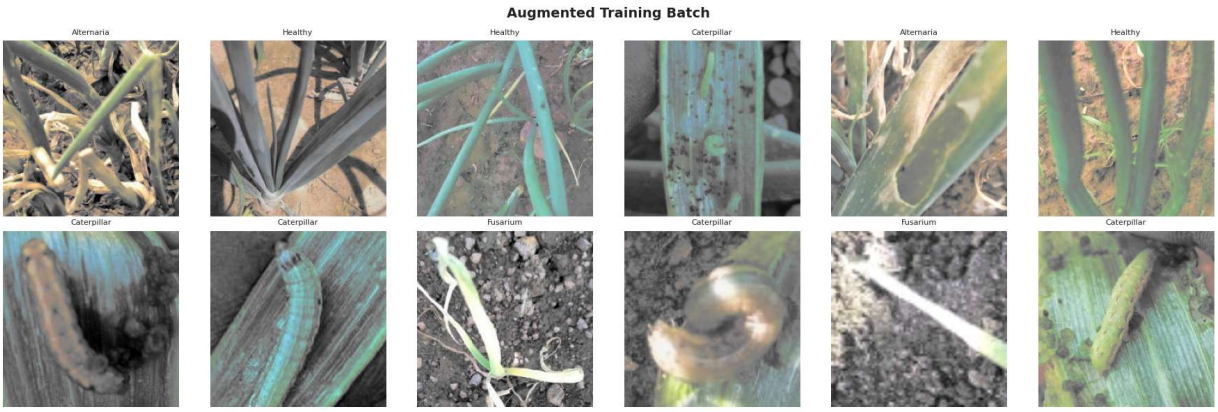
In [160…

```
# ── 2.3 Preprocessing pipelines ──────────────────────────────────────
AUTOTUNE = tf.data.AUTOTUNE

def preprocess_and_augment(images, labels):
    """Normalize to [-1, 1] FIRST, then augment."""
    images = tf.cast(images, tf.float32)
    images = images / 127.5 - 1.0
    images = tf.map_fn(augment_image, images)
    return images, labels

def preprocess_eval(images, labels):
    """Normalize to [-1, 1] only."""
    images = tf.cast(images, tf.float32)
    images = images / 127.5 - 1.0
    return images, labels

# Build pipelines — .cache() on val/test so they survive multiple epochs
train_pipeline = (
    train_ds
    .map(preprocess_and_augment, num_parallel_calls=AUTOTUNE)
    .prefetch(AUTOTUNE)
)

val_pipeline = (
    val_ds
    .cache()
    .map(preprocess_eval, num_parallel_calls=AUTOTUNE)
    .prefetch(AUTOTUNE)
)

test_pipeline = (
    test_ds
    .cache()
    .map(preprocess_eval, num_parallel_calls=AUTOTUNE)
    .prefetch(AUTOTUNE)
)

# Verify pixel range
for imgs, lbls in train_pipeline.take(1):
    print(f"Batch shape: {imgs.shape}, Labels shape: {lbls.shape}")
    print(f"Pixel range: [{imgs.numpy().min():.3f}, {imgs.numpy().max():.3f}]")
    unique, counts = np.unique(np.argmax(lbls.numpy(), axis=1), return_counts=True)
    print(f"Labels in batch: {dict(zip([CLASS_NAMES[u] for u in unique], counts))}")
    if imgs.numpy().min() >= -1.5 and imgs.numpy().max() <= 1.5:
        print("Preprocessing OK")
    else:
        print("WARNING: Pixel range is WRONG!")
```

```
Batch shape: (32, 224, 224, 3), Labels shape: (32, 6)
Pixel range: [-1.182, 1.273]
Labels in batch: {'Alternaria': np.int64(6), 'Caterpillar': np.int64(10), 'Fusariu
m': np.int64(7), 'Healthy': np.int64(8), 'Virosis': np.int64(1)}
Preprocessing OK
```

In [161…

```
# ── 2.4 Visualize augmented training batch ──────────────────────
fig, axes = plt.subplots(2, 6, figsize=(18, 6))
fig.suptitle("Augmented Training Batch", fontsize=14, fontweight="bold")

for imgs, lbls in train_pipeline.take(1):
    for i, ax in enumerate(axes.flat):
        if i < len(imgs):
            img_display = (imgs[i].numpy() + 1.0) / 2.0
            img_display = np.clip(img_display, 0, 1)
            ax.imshow(img_display)
            label_idx = np.argmax(lbls[i].numpy())
            ax.set_title(f"{CLASS_NAMES[label_idx]}", fontsize=8)
        ax.axis("off")

plt.tight_layout()
fig.savefig(OUT_DIR / "08_augmented_batch.png", dpi=150)
plt.show()
```



Part 3 — Model Training (3-Stage Progressive Fine-Tuning)

Stage	Layers Trained	Learning Rate	Epochs
Phase 1	Classifier head only	1e-3	15
Phase 2	Last ~30% of backbone + head	1e-4	20
Phase 3	All layers	1e-5	10

Optimizer: AdamW (weight_decay=0.01)

Loss: Categorical cross-entropy with label smoothing (0.1)

LR Schedule: Cosine annealing with 5-epoch linear warmup

In [162…

```
# ── 3.1 Build MobileNetV3-Large model (6-class) ─────────────────
base_model = MobileNetV3Large(
    input_shape=(IMG_SIZE, IMG_SIZE, 3),
    include_top=False,
    weights="imagenet",
```

```
    include_preprocessing=False,
)

# Freeze all base layers initially
base_model.trainable = False

# Custom classification head
inputs = keras.Input(shape=(IMG_SIZE, IMG_SIZE, 3))
x = base_model(inputs, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.BatchNormalization()(x)
x = layers.Dropout(0.3)(x)  # slightly higher dropout for 6 classes
outputs = layers.Dense(NUM_CLASSES, activation="softmax")(x)

model = keras.Model(inputs, outputs, name="OnionGuard_MultiClass")
model.summary(show_trainable=True, expand_nested=False)
print(f"\nTotal base layers: {len(base_model.layers)}")
print(f"Output classes: {NUM_CLASSES}")

# ── Diagnostic: verify model outputs
print("\n--- Pre-training diagnostic ---")
for imgs, lbls in val_pipeline.take(1):
    preds = model.predict(imgs[:4], verbose=0)
    print(f"Sample predictions (before training):")
    for i in range(min(4, len(imgs))):
        true_cls = CLASS_NAMES[np.argmax(lbls[i])]
        pred_cls = CLASS_NAMES[np.argmax(preds[i])]
        print(f"  True: {true_cls:20s} | Pred: {pred_cls} ({preds[i].round(3)})")
    print("(Random predictions expected before training)")
```

Model: "OnionGuard_MultiClass"

Layer (type)	Output Shape	Param #	Trai…
input_layer_11 (InputLayer)	(None, 224, 224, 3)	0	-
MobileNetV3Large (Functional)	(None, 7, 7, 960)	2,996,352	N
global_average_pooling2d_5 (GlobalAveragePooling2D)	(None, 960)	0	-
batch_normalization_5 (BatchNormalization)	(None, 960)	3,840	Y
dropout_5 (Dropout)	(None, 960)	0	-
dense_5 (Dense)	(None, 6)	5,766	Y

Total params: 3,005,958 (11.47 MB)

Trainable params: 7,686 (30.02 KB)

Non-trainable params: 2,998,272 (11.44 MB)

```
Total base layers: 186
Output classes: 6

--- Pre-training diagnostic ---
Sample predictions (before training):
  True: Alternaria           | Pred: Alternaria ([0.458 0.089 0.34  0.041 0.049 0.02
4])
  True: Fusarium             | Pred: Alternaria ([0.446 0.36  0.085 0.047 0.039 0.02
2])
  True: Caterpillar          | Pred: Healthy ([0.101 0.05  0.169 0.146 0.531 0.003])
  True: Caterpillar          | Pred: Fusarium ([0.044 0.071 0.286 0.509 0.039 0.05
2])
(Random predictions expected before training)
```

In [163…

```python
# ── 3.2 Learning rate schedule: cosine annealing with warmup ──────────────
class WarmupCosineDecay(keras.optimizers.schedules.LearningRateSchedule):
    def __init__(self, max_lr, warmup_steps, total_steps, min_lr=1e-7):
        super().__init__()
        self.max_lr = max_lr
        self.warmup_steps = warmup_steps
        self.total_steps = total_steps
        self.min_lr = min_lr

    def __call__(self, step):
        step = tf.cast(step, tf.float32)
        warmup_lr = self.max_lr * (step / tf.maximum(tf.cast(self.warmup_steps, tf.
        decay_steps = tf.cast(self.total_steps - self.warmup_steps, tf.float32)
        decay_step = tf.maximum(step - tf.cast(self.warmup_steps, tf.float32), 0.0)
        cosine_decay = 0.5 * (1.0 + tf.cos(np.pi * decay_step / tf.maximum(decay_st
        decay_lr = self.min_lr + (self.max_lr - self.min_lr) * cosine_decay
        return tf.where(step < tf.cast(self.warmup_steps, tf.float32), warmup_lr, d

    def get_config(self):
        return {"max_lr": self.max_lr, "warmup_steps": self.warmup_steps,
                "total_steps": self.total_steps, "min_lr": self.min_lr}

# Common callbacks
def get_callbacks(phase_name):
    return [
        callbacks.EarlyStopping(
            monitor="val_loss", patience=7, restore_best_weights=True, verbose=1
        ),
        callbacks.ModelCheckpoint(
            str(OUT_DIR / f"best_model_{phase_name}.keras"),
            monitor="val_accuracy", save_best_only=True, verbose=1
        ),
    ]

print("LR schedule & callbacks defined ✓")
```

```
LR schedule & callbacks defined ✓
```

In [164…

```python
# ── 3.3 Phase 1: Train classifier head only ──────────────────────────────
_ckpt_p1 = load_checkpoint("history_phase1")

if _ckpt_p1 is not None:
```

```
    model.load_weights(str(OUT_DIR / "best_model_phase1.keras"))
    class _FakeHistory:
        def __init__(self, h): self.history = h
    history_p1 = _FakeHistory(_ckpt_p1)
    print("Phase 1: LOADED FROM CHECKPOINT (skipping training)")
else:
    steps_per_epoch = tf.data.experimental.cardinality(train_pipeline).numpy()
    total_steps_p1 = steps_per_epoch * EPOCHS_PHASE1
    warmup_steps_p1 = steps_per_epoch * 3

    lr_schedule_p1 = WarmupCosineDecay(
        max_lr=1e-3, warmup_steps=warmup_steps_p1, total_steps=total_steps_p1
    )

    model.compile(
        optimizer=optimizers.AdamW(learning_rate=lr_schedule_p1, weight_decay=0.01)
        loss=keras.losses.CategoricalCrossentropy(),
        metrics=["accuracy"],
    )

    print(f"Phase 1: Training classifier head ({EPOCHS_PHASE1} epochs)")
    print(f"  Trainable params: {sum(np.prod(v.shape) for v in model.trainable_vari
    print(f"  Using class weights to handle imbalance")

    history_p1 = model.fit(
        train_pipeline,
        validation_data=val_pipeline,
        epochs=EPOCHS_PHASE1,
        class_weight=class_weight_dict,
        callbacks=get_callbacks("phase1"),
    )

    save_checkpoint("history_phase1", history_p1.history)

steps_per_epoch = tf.data.experimental.cardinality(train_pipeline).numpy()
print(f"Phase 1 best val_accuracy: {max(history_p1.history['val_accuracy']):.4f}")
```

```
Phase 1: Training classifier head (20 epochs)
  Trainable params: 7,686
  Using class weights to handle imbalance
Epoch 1/20
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 554ms/step - accuracy: 0.2363 - loss: 2.0447
Epoch 1: val_accuracy improved from -inf to 0.46053, saving model to /kaggle/workin
g/caterpillar_output/best_model_phase1.keras
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/epoch_iterator.py:160: Us
erWarning: Your input ran out of data; interrupting training. Make sure that your da
taset or generator can generate at least `steps_per_epoch * epochs` batches. You may
need to use the `.repeat()` function when building your dataset.
  self._interrupted_warning()
```

```
77/77 ━━━━━━━━━━━━━━━━━━━━ 60s 670ms/step - accuracy: 0.2373 - loss: 2.0410 - val_ac
curacy: 0.4605 - val_loss: 1.2676
Epoch 2/20
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 535ms/step - accuracy: 0.5766 - loss: 1.0518
Epoch 2: val_accuracy improved from 0.46053 to 0.76645, saving model to /kaggle/work
ing/caterpillar_output/best_model_phase1.keras
77/77 ━━━━━━━━━━━━━━━━━━━━ 47s 606ms/step - accuracy: 0.5775 - loss: 1.0495 - val_ac
curacy: 0.7664 - val_loss: 0.6377
Epoch 3/20
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 533ms/step - accuracy: 0.7649 - loss: 0.5829
Epoch 3: val_accuracy improved from 0.76645 to 0.83553, saving model to /kaggle/work
ing/caterpillar_output/best_model_phase1.keras
77/77 ━━━━━━━━━━━━━━━━━━━━ 82s 604ms/step - accuracy: 0.7652 - loss: 0.5823 - val_ac
curacy: 0.8355 - val_loss: 0.4763
Epoch 4/20
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 520ms/step - accuracy: 0.8202 - loss: 0.4164
Epoch 4: val_accuracy improved from 0.83553 to 0.85197, saving model to /kaggle/work
ing/caterpillar_output/best_model_phase1.keras
77/77 ━━━━━━━━━━━━━━━━━━━━ 46s 591ms/step - accuracy: 0.8203 - loss: 0.4162 - val_ac
curacy: 0.8520 - val_loss: 0.4102
Epoch 5/20
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 523ms/step - accuracy: 0.8578 - loss: 0.3577
Epoch 5: val_accuracy improved from 0.85197 to 0.86184, saving model to /kaggle/work
ing/caterpillar_output/best_model_phase1.keras
77/77 ━━━━━━━━━━━━━━━━━━━━ 46s 597ms/step - accuracy: 0.8577 - loss: 0.3577 - val_ac
curacy: 0.8618 - val_loss: 0.3923
Epoch 6/20
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 511ms/step - accuracy: 0.8793 - loss: 0.3160
Epoch 6: val_accuracy improved from 0.86184 to 0.87829, saving model to /kaggle/work
ing/caterpillar_output/best_model_phase1.keras
77/77 ━━━━━━━━━━━━━━━━━━━━ 45s 585ms/step - accuracy: 0.8792 - loss: 0.3161 - val_ac
curacy: 0.8783 - val_loss: 0.3716
Epoch 7/20
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 601ms/step - accuracy: 0.8720 - loss: 0.3140
Epoch 7: val_accuracy did not improve from 0.87829
77/77 ━━━━━━━━━━━━━━━━━━━━ 52s 673ms/step - accuracy: 0.8719 - loss: 0.3139 - val_ac
curacy: 0.8783 - val_loss: 0.3472
Epoch 8/20
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 543ms/step - accuracy: 0.8812 - loss: 0.2707
Epoch 8: val_accuracy did not improve from 0.87829
77/77 ━━━━━━━━━━━━━━━━━━━━ 47s 608ms/step - accuracy: 0.8811 - loss: 0.2709 - val_ac
curacy: 0.8750 - val_loss: 0.3537
Epoch 9/20
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 551ms/step - accuracy: 0.8902 - loss: 0.2479
Epoch 9: val_accuracy did not improve from 0.87829
77/77 ━━━━━━━━━━━━━━━━━━━━ 48s 615ms/step - accuracy: 0.8903 - loss: 0.2480 - val_ac
curacy: 0.8651 - val_loss: 0.3559
Epoch 10/20
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 520ms/step - accuracy: 0.9004 - loss: 0.2281
Epoch 10: val_accuracy did not improve from 0.87829
77/77 ━━━━━━━━━━━━━━━━━━━━ 46s 589ms/step - accuracy: 0.9004 - loss: 0.2282 - val_ac
curacy: 0.8783 - val_loss: 0.3450
Epoch 11/20
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 499ms/step - accuracy: 0.9078 - loss: 0.2282
Epoch 11: val_accuracy improved from 0.87829 to 0.88158, saving model to /kaggle/wor
king/caterpillar_output/best_model_phase1.keras
```

```
77/77 ━━━━━━━━━━━━━━━━━━━━ 45s 582ms/step - accuracy: 0.9077 - loss: 0.2283 - val_ac
curacy: 0.8816 - val_loss: 0.3453
Epoch 12/20
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 497ms/step - accuracy: 0.9003 - loss: 0.2250
Epoch 12: val_accuracy did not improve from 0.88158
77/77 ━━━━━━━━━━━━━━━━━━━━ 43s 558ms/step - accuracy: 0.9002 - loss: 0.2251 - val_ac
curacy: 0.8783 - val_loss: 0.3492
Epoch 13/20
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 576ms/step - accuracy: 0.8963 - loss: 0.2398
Epoch 13: val_accuracy improved from 0.88158 to 0.89145, saving model to /kaggle/wor
king/caterpillar_output/best_model_phase1.keras
77/77 ━━━━━━━━━━━━━━━━━━━━ 54s 703ms/step - accuracy: 0.8964 - loss: 0.2397 - val_ac
curacy: 0.8914 - val_loss: 0.3401
Epoch 14/20
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 616ms/step - accuracy: 0.9086 - loss: 0.2253
Epoch 14: val_accuracy did not improve from 0.89145
77/77 ━━━━━━━━━━━━━━━━━━━━ 56s 713ms/step - accuracy: 0.9085 - loss: 0.2253 - val_ac
curacy: 0.8882 - val_loss: 0.3373
Epoch 15/20
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 536ms/step - accuracy: 0.9122 - loss: 0.2036
Epoch 15: val_accuracy did not improve from 0.89145
77/77 ━━━━━━━━━━━━━━━━━━━━ 47s 603ms/step - accuracy: 0.9121 - loss: 0.2037 - val_ac
curacy: 0.8816 - val_loss: 0.3399
Epoch 16/20
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 558ms/step - accuracy: 0.9083 - loss: 0.2210
Epoch 16: val_accuracy did not improve from 0.89145
77/77 ━━━━━━━━━━━━━━━━━━━━ 49s 628ms/step - accuracy: 0.9083 - loss: 0.2208 - val_ac
curacy: 0.8849 - val_loss: 0.3351
Epoch 17/20
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 572ms/step - accuracy: 0.9145 - loss: 0.2102
Epoch 17: val_accuracy did not improve from 0.89145
77/77 ━━━━━━━━━━━━━━━━━━━━ 85s 663ms/step - accuracy: 0.9144 - loss: 0.2102 - val_ac
curacy: 0.8849 - val_loss: 0.3338
Epoch 18/20
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 629ms/step - accuracy: 0.9113 - loss: 0.2088
Epoch 18: val_accuracy did not improve from 0.89145
77/77 ━━━━━━━━━━━━━━━━━━━━ 55s 711ms/step - accuracy: 0.9112 - loss: 0.2088 - val_ac
curacy: 0.8849 - val_loss: 0.3347
Epoch 19/20
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 557ms/step - accuracy: 0.9127 - loss: 0.2199
Epoch 19: val_accuracy did not improve from 0.89145
77/77 ━━━━━━━━━━━━━━━━━━━━ 48s 620ms/step - accuracy: 0.9127 - loss: 0.2199 - val_ac
curacy: 0.8849 - val_loss: 0.3351
Epoch 20/20
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 517ms/step - accuracy: 0.9007 - loss: 0.2138
Epoch 20: val_accuracy did not improve from 0.89145
77/77 ━━━━━━━━━━━━━━━━━━━━ 45s 581ms/step - accuracy: 0.9007 - loss: 0.2138 - val_ac
curacy: 0.8849 - val_loss: 0.3347
Restoring model weights from the end of the best epoch: 17.
  [CHECKPOINT SAVED] history_phase1
Phase 1 best val_accuracy: 0.8914
```

In [165…

```python
# ── 3.4 Phase 2: Unfreeze last ~30% of backbone ──────────────────────
_ckpt_p2 = load_checkpoint("history_phase2")

if _ckpt_p2 is not None:
```

```
    model.load_weights(str(OUT_DIR / "best_model_phase2.keras"))
    # Ensure layers are unfrozen to correct state
    base_model.trainable = True
    num_layers = len(base_model.layers)
    fine_tune_from = int(num_layers * 0.7)
    for layer in base_model.layers[:fine_tune_from]:
        layer.trainable = False
    class _FakeHistory:
        def __init__(self, h): self.history = h
    history_p2 = _FakeHistory(_ckpt_p2)
    print("Phase 2: LOADED FROM CHECKPOINT (skipping training)")
else:
    base_model.trainable = True
    num_layers = len(base_model.layers)
    fine_tune_from = int(num_layers * 0.7)

    for layer in base_model.layers[:fine_tune_from]:
        layer.trainable = False

    trainable_count = sum(1 for l in base_model.layers if l.trainable)
    print(f"Phase 2: Unfroze {trainable_count}/{num_layers} backbone layers (from l

    total_steps_p2 = steps_per_epoch * EPOCHS_PHASE2
    warmup_steps_p2 = steps_per_epoch * 3

    lr_schedule_p2 = WarmupCosineDecay(
        max_lr=1e-4, warmup_steps=warmup_steps_p2, total_steps=total_steps_p2
    )

    model.compile(
        optimizer=optimizers.AdamW(learning_rate=lr_schedule_p2, weight_decay=0.01)
        loss=keras.losses.CategoricalCrossentropy(),
        metrics=["accuracy"],
    )

    print(f"  Trainable params: {sum(np.prod(v.shape) for v in model.trainable_vari

    history_p2 = model.fit(
        train_pipeline,
        validation_data=val_pipeline,
        epochs=EPOCHS_PHASE2,
        class_weight=class_weight_dict,
        callbacks=get_callbacks("phase2"),
    )

    save_checkpoint("history_phase2", history_p2.history)

print(f"Phase 2 best val_accuracy: {max(history_p2.history['val_accuracy']):.4f}")
```

```
Phase 2: Unfroze 56/186 backbone layers (from layer 130)
  Trainable params: 2,489,270
Epoch 1/25
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 815ms/step - accuracy: 0.8615 - loss: 0.3163
Epoch 1: val_accuracy improved from -inf to 0.84539, saving model to /kaggle/workin
g/caterpillar_output/best_model_phase2.keras
77/77 ━━━━━━━━━━━━━━━━━━━━ 90s 921ms/step - accuracy: 0.8616 - loss: 0.3162 - val_ac
curacy: 0.8454 - val_loss: 0.3804
Epoch 2/25
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 807ms/step - accuracy: 0.8797 - loss: 0.2999
Epoch 2: val_accuracy did not improve from 0.84539
77/77 ━━━━━━━━━━━━━━━━━━━━ 67s 869ms/step - accuracy: 0.8798 - loss: 0.2994 - val_ac
curacy: 0.8125 - val_loss: 0.4357
Epoch 3/25
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 820ms/step - accuracy: 0.9044 - loss: 0.2261
Epoch 3: val_accuracy did not improve from 0.84539
77/77 ━━━━━━━━━━━━━━━━━━━━ 68s 885ms/step - accuracy: 0.9044 - loss: 0.2260 - val_ac
curacy: 0.8092 - val_loss: 0.4586
Epoch 4/25
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 829ms/step - accuracy: 0.9355 - loss: 0.1375
Epoch 4: val_accuracy did not improve from 0.84539
77/77 ━━━━━━━━━━━━━━━━━━━━ 69s 892ms/step - accuracy: 0.9355 - loss: 0.1376 - val_ac
curacy: 0.8322 - val_loss: 0.4193
Epoch 5/25
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 810ms/step - accuracy: 0.9383 - loss: 0.1378
Epoch 5: val_accuracy did not improve from 0.84539
77/77 ━━━━━━━━━━━━━━━━━━━━ 67s 870ms/step - accuracy: 0.9384 - loss: 0.1376 - val_ac
curacy: 0.8257 - val_loss: 0.4373
Epoch 6/25
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 786ms/step - accuracy: 0.9556 - loss: 0.1085
Epoch 6: val_accuracy did not improve from 0.84539
77/77 ━━━━━━━━━━━━━━━━━━━━ 66s 846ms/step - accuracy: 0.9557 - loss: 0.1084 - val_ac
curacy: 0.8421 - val_loss: 0.4266
Epoch 7/25
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 771ms/step - accuracy: 0.9772 - loss: 0.0719
Epoch 7: val_accuracy improved from 0.84539 to 0.85526, saving model to /kaggle/work
ing/caterpillar_output/best_model_phase2.keras
77/77 ━━━━━━━━━━━━━━━━━━━━ 65s 840ms/step - accuracy: 0.9771 - loss: 0.0719 - val_ac
curacy: 0.8553 - val_loss: 0.3849
Epoch 8/25
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 764ms/step - accuracy: 0.9815 - loss: 0.0665
Epoch 8: val_accuracy improved from 0.85526 to 0.85855, saving model to /kaggle/work
ing/caterpillar_output/best_model_phase2.keras
77/77 ━━━━━━━━━━━━━━━━━━━━ 65s 833ms/step - accuracy: 0.9815 - loss: 0.0664 - val_ac
curacy: 0.8586 - val_loss: 0.3748
Epoch 9/25
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 785ms/step - accuracy: 0.9786 - loss: 0.0538
Epoch 9: val_accuracy improved from 0.85855 to 0.88487, saving model to /kaggle/work
ing/caterpillar_output/best_model_phase2.keras
77/77 ━━━━━━━━━━━━━━━━━━━━ 82s 1s/step - accuracy: 0.9786 - loss: 0.0537 - val_accur
acy: 0.8849 - val_loss: 0.3350
Epoch 10/25
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 760ms/step - accuracy: 0.9822 - loss: 0.0503
Epoch 10: val_accuracy improved from 0.88487 to 0.89145, saving model to /kaggle/wor
king/caterpillar_output/best_model_phase2.keras
77/77 ━━━━━━━━━━━━━━━━━━━━ 64s 829ms/step - accuracy: 0.9822 - loss: 0.0502 - val_ac
```

```
curacy: 0.8914 - val_loss: 0.3425
Epoch 11/25
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 771ms/step - accuracy: 0.9870 - loss: 0.0405
Epoch 11: val_accuracy did not improve from 0.89145
77/77 ━━━━━━━━━━━━━━━━━━━━ 64s 831ms/step - accuracy: 0.9870 - loss: 0.0405 - val_ac
curacy: 0.8914 - val_loss: 0.3320
Epoch 12/25
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 813ms/step - accuracy: 0.9872 - loss: 0.0383
Epoch 12: val_accuracy improved from 0.89145 to 0.90132, saving model to /kaggle/wor
king/caterpillar_output/best_model_phase2.keras
77/77 ━━━━━━━━━━━━━━━━━━━━ 70s 900ms/step - accuracy: 0.9872 - loss: 0.0383 - val_ac
curacy: 0.9013 - val_loss: 0.3243
Epoch 13/25
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 975ms/step - accuracy: 0.9913 - loss: 0.0303
Epoch 13: val_accuracy did not improve from 0.90132
77/77 ━━━━━━━━━━━━━━━━━━━━ 81s 1s/step - accuracy: 0.9913 - loss: 0.0303 - val_accur
acy: 0.9013 - val_loss: 0.3212
Epoch 14/25
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 809ms/step - accuracy: 0.9917 - loss: 0.0280
Epoch 14: val_accuracy improved from 0.90132 to 0.91118, saving model to /kaggle/wor
king/caterpillar_output/best_model_phase2.keras
77/77 ━━━━━━━━━━━━━━━━━━━━ 70s 885ms/step - accuracy: 0.9917 - loss: 0.0280 - val_ac
curacy: 0.9112 - val_loss: 0.3049
Epoch 15/25
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 811ms/step - accuracy: 0.9866 - loss: 0.0361
Epoch 15: val_accuracy did not improve from 0.91118
77/77 ━━━━━━━━━━━━━━━━━━━━ 68s 876ms/step - accuracy: 0.9866 - loss: 0.0360 - val_ac
curacy: 0.9079 - val_loss: 0.3008
Epoch 16/25
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 818ms/step - accuracy: 0.9917 - loss: 0.0283
Epoch 16: val_accuracy did not improve from 0.91118
77/77 ━━━━━━━━━━━━━━━━━━━━ 82s 880ms/step - accuracy: 0.9917 - loss: 0.0283 - val_ac
curacy: 0.9079 - val_loss: 0.2965
Epoch 17/25
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 774ms/step - accuracy: 0.9960 - loss: 0.0210
Epoch 17: val_accuracy improved from 0.91118 to 0.92105, saving model to /kaggle/wor
king/caterpillar_output/best_model_phase2.keras
77/77 ━━━━━━━━━━━━━━━━━━━━ 66s 848ms/step - accuracy: 0.9960 - loss: 0.0211 - val_ac
curacy: 0.9211 - val_loss: 0.2852
Epoch 18/25
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 790ms/step - accuracy: 0.9970 - loss: 0.0166
Epoch 18: val_accuracy improved from 0.92105 to 0.92434, saving model to /kaggle/wor
king/caterpillar_output/best_model_phase2.keras
77/77 ━━━━━━━━━━━━━━━━━━━━ 67s 861ms/step - accuracy: 0.9969 - loss: 0.0166 - val_ac
curacy: 0.9243 - val_loss: 0.2800
Epoch 19/25
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 768ms/step - accuracy: 0.9964 - loss: 0.0207
Epoch 19: val_accuracy improved from 0.92434 to 0.93092, saving model to /kaggle/wor
king/caterpillar_output/best_model_phase2.keras
77/77 ━━━━━━━━━━━━━━━━━━━━ 65s 838ms/step - accuracy: 0.9964 - loss: 0.0207 - val_ac
curacy: 0.9309 - val_loss: 0.2776
Epoch 20/25
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 785ms/step - accuracy: 0.9946 - loss: 0.0183
Epoch 20: val_accuracy did not improve from 0.93092
77/77 ━━━━━━━━━━━━━━━━━━━━ 66s 850ms/step - accuracy: 0.9946 - loss: 0.0183 - val_ac
curacy: 0.9309 - val_loss: 0.2723
```

```
Epoch 21/25
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 822ms/step - accuracy: 0.9953 - loss: 0.0246
Epoch 21: val_accuracy did not improve from 0.93092
77/77 ━━━━━━━━━━━━━━━━━━━━ 69s 891ms/step - accuracy: 0.9953 - loss: 0.0245 - val_ac
curacy: 0.9276 - val_loss: 0.2695
Epoch 22/25
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 800ms/step - accuracy: 0.9930 - loss: 0.0229
Epoch 22: val_accuracy did not improve from 0.93092
77/77 ━━━━━━━━━━━━━━━━━━━━ 67s 861ms/step - accuracy: 0.9930 - loss: 0.0229 - val_ac
curacy: 0.9276 - val_loss: 0.2649
Epoch 23/25
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 821ms/step - accuracy: 0.9953 - loss: 0.0195
Epoch 23: val_accuracy did not improve from 0.93092
77/77 ━━━━━━━━━━━━━━━━━━━━ 68s 884ms/step - accuracy: 0.9953 - loss: 0.0196 - val_ac
curacy: 0.9309 - val_loss: 0.2642
Epoch 24/25
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 776ms/step - accuracy: 0.9952 - loss: 0.0172
Epoch 24: val_accuracy did not improve from 0.93092
77/77 ━━━━━━━━━━━━━━━━━━━━ 79s 839ms/step - accuracy: 0.9951 - loss: 0.0172 - val_ac
curacy: 0.9309 - val_loss: 0.2604
Epoch 25/25
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 796ms/step - accuracy: 0.9946 - loss: 0.0216
Epoch 25: val_accuracy did not improve from 0.93092
77/77 ━━━━━━━━━━━━━━━━━━━━ 67s 860ms/step - accuracy: 0.9946 - loss: 0.0216 - val_ac
curacy: 0.9309 - val_loss: 0.2581
Restoring model weights from the end of the best epoch: 25.
  [CHECKPOINT SAVED] history_phase2
Phase 2 best val_accuracy: 0.9309
```

In [166…

```python
# ── 3.5 Phase 3: Unfreeze ALL layers (full fine-tuning) ──────────────
_ckpt_p3 = load_checkpoint("history_phase3")

if _ckpt_p3 is not None:
    model.load_weights(str(OUT_DIR / "best_model_phase3.keras"))
    for layer in base_model.layers:
        layer.trainable = True
    class _FakeHistory:
        def __init__(self, h): self.history = h
    history_p3 = _FakeHistory(_ckpt_p3)
    print("Phase 3: LOADED FROM CHECKPOINT (skipping training)")
else:
    for layer in base_model.layers:
        layer.trainable = True

    total_steps_p3 = steps_per_epoch * EPOCHS_PHASE3
    warmup_steps_p3 = steps_per_epoch * 2

    lr_schedule_p3 = WarmupCosineDecay(
        max_lr=1e-5, warmup_steps=warmup_steps_p3, total_steps=total_steps_p3
    )

    model.compile(
        optimizer=optimizers.AdamW(learning_rate=lr_schedule_p3, weight_decay=0.01),
        loss=keras.losses.CategoricalCrossentropy(),
        metrics=["accuracy"],
    )
```

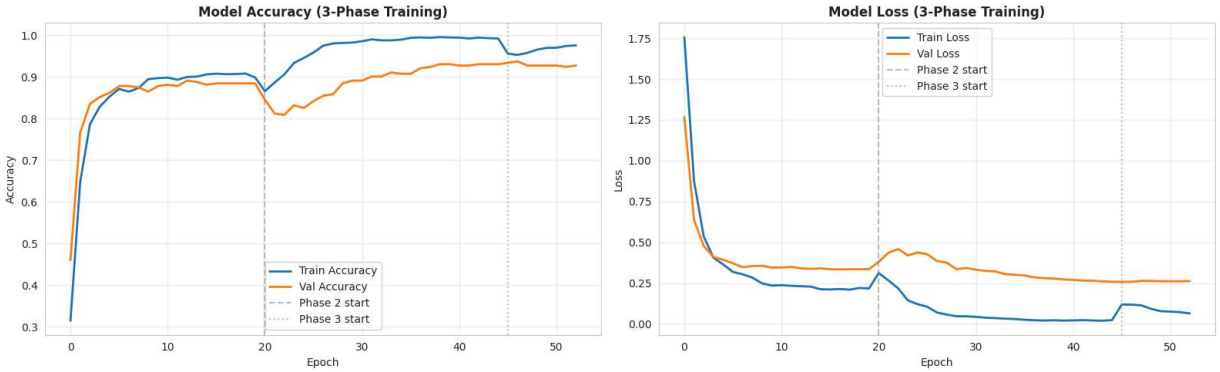
```
    print(f"Phase 3: Full fine-tuning — ALL layers trainable ({EPOCHS_PHASE3} epoch
    print(f"  Trainable params: {sum(np.prod(v.shape) for v in model.trainable_vari

    history_p3 = model.fit(
        train_pipeline,
        validation_data=val_pipeline,
        epochs=EPOCHS_PHASE3,
        class_weight=class_weight_dict,
        callbacks=get_callbacks("phase3"),
    )

    save_checkpoint("history_phase3", history_p3.history)

print(f"Phase 3 best val_accuracy: {max(history_p3.history['val_accuracy']):.4f}")
```

```
Phase 3: Full fine-tuning — ALL layers trainable (15 epochs)
  Trainable params: 2,979,638
Epoch 1/15
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 3s/step - accuracy: 0.9548 - loss: 0.1189
Epoch 1: val_accuracy improved from -inf to 0.93421, saving model to /kaggle/workin
g/caterpillar_output/best_model_phase3.keras
77/77 ━━━━━━━━━━━━━━━━━━━━ 271s 3s/step - accuracy: 0.9549 - loss: 0.1189 - val_accu
racy: 0.9342 - val_loss: 0.2574
Epoch 2/15
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.9508 - loss: 0.1234
Epoch 2: val_accuracy improved from 0.93421 to 0.93750, saving model to /kaggle/work
ing/caterpillar_output/best_model_phase3.keras
77/77 ━━━━━━━━━━━━━━━━━━━━ 196s 3s/step - accuracy: 0.9508 - loss: 0.1233 - val_accu
racy: 0.9375 - val_loss: 0.2579
Epoch 3/15
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 3s/step - accuracy: 0.9628 - loss: 0.1050
Epoch 3: val_accuracy did not improve from 0.93750
77/77 ━━━━━━━━━━━━━━━━━━━━ 215s 3s/step - accuracy: 0.9628 - loss: 0.1051 - val_accu
racy: 0.9276 - val_loss: 0.2636
Epoch 4/15
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.9627 - loss: 0.0970
Epoch 4: val_accuracy did not improve from 0.93750
77/77 ━━━━━━━━━━━━━━━━━━━━ 196s 3s/step - accuracy: 0.9627 - loss: 0.0969 - val_accu
racy: 0.9276 - val_loss: 0.2628
Epoch 5/15
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.9688 - loss: 0.0818
Epoch 5: val_accuracy did not improve from 0.93750
77/77 ━━━━━━━━━━━━━━━━━━━━ 195s 3s/step - accuracy: 0.9688 - loss: 0.0817 - val_accu
racy: 0.9276 - val_loss: 0.2613
Epoch 6/15
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 3s/step - accuracy: 0.9684 - loss: 0.0797
Epoch 6: val_accuracy did not improve from 0.93750
77/77 ━━━━━━━━━━━━━━━━━━━━ 205s 3s/step - accuracy: 0.9684 - loss: 0.0797 - val_accu
racy: 0.9276 - val_loss: 0.2612
Epoch 7/15
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.9731 - loss: 0.0825
Epoch 7: val_accuracy did not improve from 0.93750
77/77 ━━━━━━━━━━━━━━━━━━━━ 194s 3s/step - accuracy: 0.9732 - loss: 0.0824 - val_accu
racy: 0.9243 - val_loss: 0.2607
Epoch 8/15
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 3s/step - accuracy: 0.9734 - loss: 0.0673
Epoch 8: val_accuracy did not improve from 0.93750
77/77 ━━━━━━━━━━━━━━━━━━━━ 204s 3s/step - accuracy: 0.9735 - loss: 0.0673 - val_accu
racy: 0.9276 - val_loss: 0.2616
Epoch 8: early stopping
Restoring model weights from the end of the best epoch: 1.
  [CHECKPOINT SAVED] history_phase3
Phase 3 best val_accuracy: 0.9375
```

In [167…

```python
# ── 3.6 Training history — combined plot ──────────────────────────────
def merge_histories(*histories):
    merged = {}
    for h in histories:
        for key, vals in h.history.items():
            merged.setdefault(key, []).extend(vals)
    return merged
```

```python
history_all = merge_histories(history_p1, history_p2, history_p3)

fig, axes = plt.subplots(1, 2, figsize=(16, 5))

# Accuracy
axes[0].plot(history_all["accuracy"], label="Train Accuracy", linewidth=2)
axes[0].plot(history_all["val_accuracy"], label="Val Accuracy", linewidth=2)
# Mark phase boundaries
p1_end = len(history_p1.history["accuracy"])
p2_end = p1_end + len(history_p2.history["accuracy"])
axes[0].axvline(x=p1_end, color="gray", linestyle="--", alpha=0.5, label="Phase 2 s
axes[0].axvline(x=p2_end, color="gray", linestyle=":", alpha=0.5, label="Phase 3 st
axes[0].set_title("Model Accuracy (3-Phase Training)", fontweight="bold")
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Accuracy")
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Loss
axes[1].plot(history_all["loss"], label="Train Loss", linewidth=2)
axes[1].plot(history_all["val_loss"], label="Val Loss", linewidth=2)
axes[1].axvline(x=p1_end, color="gray", linestyle="--", alpha=0.5, label="Phase 2 s
axes[1].axvline(x=p2_end, color="gray", linestyle=":", alpha=0.5, label="Phase 3 st
axes[1].set_title("Model Loss (3-Phase Training)", fontweight="bold")
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Loss")
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
fig.savefig(OUT_DIR / "09_training_history.png", dpi=150)
plt.show()

best_val_acc = max(history_all["val_accuracy"])
print(f"\nBest validation accuracy: {best_val_acc:.4f} ({best_val_acc*100:.2f}%)")
```




```
Best validation accuracy: 0.9375 (93.75%)
```

Part 4 — Evaluation on Test Set

Confusion matrix, classification report, and per-class accuracy on the held-out Category B test set.

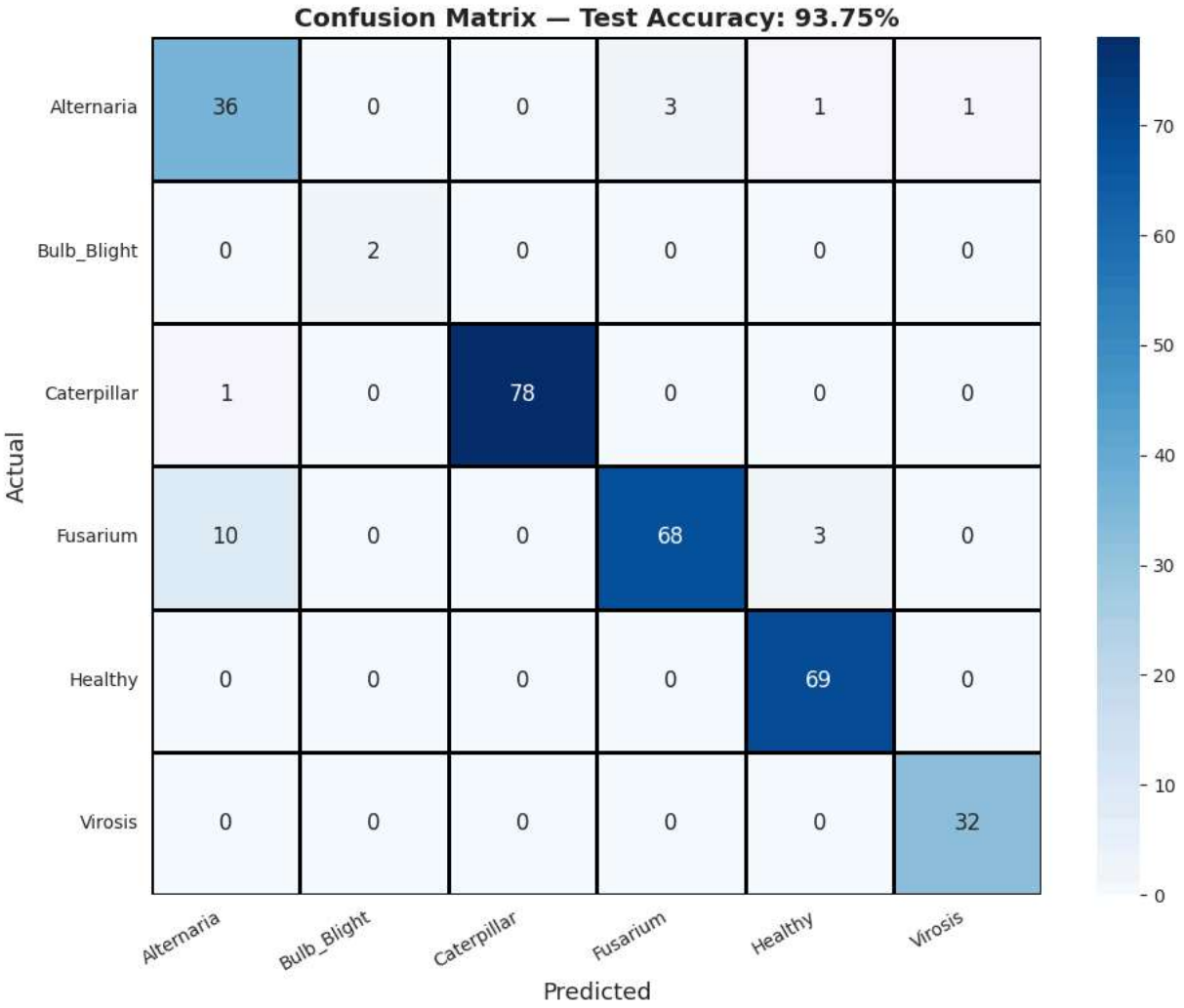
In [168…

```
# ── 4.1 Test set evaluation ──────────────────────────────────────
model.compile(loss=keras.losses.CategoricalCrossentropy(), metrics=["accuracy"])
test_loss, test_acc = model.evaluate(test_pipeline, verbose=1)
print(f"\nTest Accuracy: {test_acc:.4f} ({test_acc*100:.2f}%)")
print(f"Test Loss:     {test_loss:.4f}")

# Get predictions
y_true = []
y_pred = []
for imgs, lbls in test_pipeline:
    preds = model.predict(imgs, verbose=0)
    y_true.extend(np.argmax(lbls.numpy(), axis=1))
    y_pred.extend(np.argmax(preds, axis=1))

y_true = np.array(y_true)
y_pred = np.array(y_pred)

# Prediction distribution
display_names = CLASS_NAMES
print(f"\nPrediction distribution:")
for i, name in enumerate(display_names):
    pred_count = np.sum(y_pred == i)
    true_count = np.sum(y_true == i)
    print(f"  {name:20s}: pred={pred_count:4d}, true={true_count:4d}")

print("\n" + "=" * 60)
print("CLASSIFICATION REPORT")
print("=" * 60)
print(classification_report(
    y_true, y_pred,
    target_names=display_names,
    labels=list(range(NUM_CLASSES)),
    digits=4,
    zero_division=0
))

save_checkpoint("evaluation", {
    "test_acc": test_acc, "test_loss": test_loss,
    "y_true": y_true, "y_pred": y_pred,
})
```

```
10/10 ━━━━━━━━━━━━━━━━━━━━ 9s 451ms/step - accuracy: 0.9403 - loss: 0.1819

Test Accuracy: 0.9375 (93.75%)
Test Loss:     0.1866

Prediction distribution:
  Alternaria          : pred=  47, true=  41
  Bulb_Blight         : pred=   2, true=   2
  Caterpillar         : pred=  78, true=  79
  Fusarium            : pred=  71, true=  81
  Healthy             : pred=  73, true=  69
  Virosis             : pred=  33, true=  32

============================================================
CLASSIFICATION REPORT
============================================================
              precision    recall  f1-score   support

  Alternaria     0.7660    0.8780    0.8182        41
 Bulb_Blight     1.0000    1.0000    1.0000         2
 Caterpillar     1.0000    0.9873    0.9936        79
    Fusarium     0.9577    0.8395    0.8947        81
     Healthy     0.9452    1.0000    0.9718        69
     Virosis     0.9697    1.0000    0.9846        32

    accuracy                         0.9375       304
   macro avg     0.9398    0.9508    0.9438       304
weighted avg     0.9416    0.9375    0.9378       304

  [CHECKPOINT SAVED] evaluation
```

In [169…

```python
# ── 4.2 Confusion matrix ──────────────────────────────────────
cm = confusion_matrix(y_true, y_pred, labels=list(range(NUM_CLASSES)))

fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=display_names, yticklabels=display_names,
            annot_kws={"size": 12}, linewidths=1, linecolor="black")
ax.set_xlabel("Predicted", fontsize=13)
ax.set_ylabel("Actual", fontsize=13)
ax.set_title(f"Confusion Matrix — Test Accuracy: {test_acc*100:.2f}%",
             fontsize=14, fontweight="bold")
plt.xticks(rotation=30, ha="right")
plt.yticks(rotation=0)
plt.tight_layout()
fig.savefig(OUT_DIR / "10_confusion_matrix.png", dpi=150)
plt.show()

# Per-class accuracy
print("\nPer-class accuracy:")
for i, name in enumerate(display_names):
    class_total = cm[i].sum()
    class_correct = cm[i][i] if class_total > 0 else 0
    acc = class_correct / class_total * 100 if class_total > 0 else 0
    print(f"  {name:20s}: {class_correct}/{class_total} = {acc:.2f}%")
```




```
Per-class accuracy:
  Alternaria          : 36/41 = 87.80%
  Bulb_Blight         : 2/2 = 100.00%
  Caterpillar         : 78/79 = 98.73%
  Fusarium            : 68/81 = 83.95%
  Healthy             : 69/69 = 100.00%
  Virosis             : 32/32 = 100.00%
```

In [170…

```python
# ── 4.3 Misclassified samples ──────────────────────────────────────
misclassified_idx = np.where(y_true != y_pred)[0]
print(f"Misclassified: {len(misclassified_idx)} / {len(y_true)} ({len(misclassified

if len(misclassified_idx) > 0:
    # Collect misclassified images
    all_test_imgs = []
    for imgs, _ in test_pipeline:
        all_test_imgs.append(imgs.numpy())
    all_test_imgs = np.concatenate(all_test_imgs, axis=0)

    show_n = min(12, len(misclassified_idx))
    sample_idx = np.random.choice(misclassified_idx, show_n, replace=False)

    fig, axes = plt.subplots(2, 6, figsize=(18, 6))
    fig.suptitle("Misclassified Samples", fontsize=14, fontweight="bold")
```

```
    for i, ax in enumerate(axes.flat):
        if i < show_n:
            idx = sample_idx[i]
            ax.imshow(all_test_imgs[idx])
            ax.set_title(f"True: {display_names[y_true[idx]]}\nPred: {display_names
                         fontsize=8, color="red")
        ax.axis("off")

    plt.tight_layout()
    fig.savefig(OUT_DIR / "11_misclassified.png", dpi=150)
    plt.show()
else:
    print("No misclassifications! Perfect test accuracy.")
```

```
Misclassified: 19 / 304 (6.25%)
```

```
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB
data ([0..1] for floats or [0..255] for integers). Got range [-0.9901909..1.0].
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB
data ([0..1] for floats or [0..255] for integers). Got range [-1.0..1.0].
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB
data ([0..1] for floats or [0..255] for integers). Got range [-1.0..0.79587674].
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB
data ([0..1] for floats or [0..255] for integers). Got range [-0.8410553..1.0].
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB
data ([0..1] for floats or [0..255] for integers). Got range [-1.0..1.0].
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB
data ([0..1] for floats or [0..255] for integers). Got range [-1.0..1.0].
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB
data ([0..1] for floats or [0..255] for integers). Got range [-0.77080846..1.0].
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB
data ([0..1] for floats or [0..255] for integers). Got range [-0.9852598..1.0].
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB
data ([0..1] for floats or [0..255] for integers). Got range [-1.0..1.0].
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB
data ([0..1] for floats or [0..255] for integers). Got range [-0.9578624..1.0].
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB
data ([0..1] for floats or [0..255] for integers). Got range [-1.0..1.0].
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB
data ([0..1] for floats or [0..255] for integers). Got range [-1.0..1.0].
```

Part 5 — Grad-CAM Interpretability

Visualize what regions of the image the model focuses on when making predictions. Critical for building farmer trust in AI recommendations.

In [171…

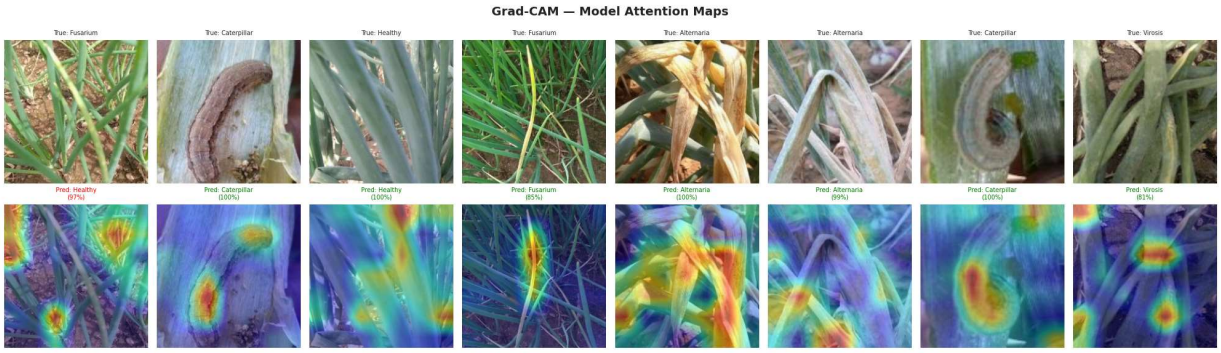
```
# ── 5.1 Grad-CAM implementation ──────────────────────────────────
def get_gradcam_heatmap(model, img_array, pred_index=None):
    """Generate Grad-CAM heatmap by manually splitting the forward pass."""
    img_tensor = tf.cast(img_array, tf.float32)

    # Split forward pass: base_model -> head layers
    with tf.GradientTape() as tape:
        # Step 1: Get conv features from base model
        conv_outputs = base_model(img_tensor, training=False)
        tape.watch(conv_outputs)

        # Step 2: Manually pass through head layers
        x = conv_outputs
        for layer in model.layers:
            if layer.name == base_model.name:
                continue
            if 'input' in layer.name:
                continue
            x = layer(x)

        predictions = x
        if pred_index is None:
            pred_index = tf.argmax(predictions[0])
        class_channel = predictions[:, pred_index]

    grads = tape.gradient(class_channel, conv_outputs)

    if grads is None:
        # Fallback: input-gradient saliency map
        with tf.GradientTape() as tape2:
            tape2.watch(img_tensor)
            preds = model(img_tensor)
            loss = preds[:, tf.argmax(preds[0])]
        input_grads = tape2.gradient(loss, img_tensor)
        if input_grads is not None:
            heatmap = tf.reduce_mean(tf.abs(input_grads[0]), axis=-1)
            heatmap = tf.maximum(heatmap, 0) / (tf.math.reduce_max(heatmap) + 1e-8)
            return heatmap.numpy()
        return np.zeros((7, 7), dtype=np.float32)

    pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))
    conv_outputs = conv_outputs[0]
    heatmap = conv_outputs @ pooled_grads[..., tf.newaxis]
    heatmap = tf.squeeze(heatmap)
    heatmap = tf.maximum(heatmap, 0) / (tf.math.reduce_max(heatmap) + 1e-8)
    return heatmap.numpy()

def overlay_gradcam(img, heatmap, alpha=0.4):
    """Overlay heatmap on original image."""
```

```
    heatmap_resized = cv2.resize(heatmap, (img.shape[1], img.shape[0]))
    heatmap_colored = cv2.applyColorMap(np.uint8(255 * heatmap_resized), cv2.COLORM
    heatmap_colored = cv2.cvtColor(heatmap_colored, cv2.COLOR_BGR2RGB) / 255.0
    overlay = alpha * heatmap_colored + (1 - alpha) * img
    return np.clip(overlay, 0, 1)

print("Grad-CAM functions defined.")
```

```
Grad-CAM functions defined.
```

In [172…

```
# ── 5.2 Grad-CAM visualizations ──────────────────────────────────
n_samples = min(8, NUM_CLASSES * 2)
fig, axes = plt.subplots(2, n_samples, figsize=(2.5 * n_samples, 6))
fig.suptitle("Grad-CAM — Model Attention Maps", fontsize=14, fontweight="bold")

sample_count = 0
for imgs, lbls in test_pipeline:
    for i in range(len(imgs)):
        if sample_count >= n_samples:
            break
        img = imgs[i:i+1]
        true_label = np.argmax(lbls[i].numpy())
        heatmap = get_gradcam_heatmap(model, img)
        pred = model.predict(img, verbose=0)
        pred_label = np.argmax(pred[0])
        confidence = pred[0][pred_label] * 100

        img_display = (img[0].numpy() + 1.0) / 2.0
        img_display = np.clip(img_display, 0, 1)

        color = "green" if pred_label == true_label else "red"
        axes[0][sample_count].imshow(img_display)
        axes[0][sample_count].set_title(f"True: {display_names[true_label]}", fonts
        axes[0][sample_count].axis("off")

        overlay = overlay_gradcam(img_display, heatmap)
        axes[1][sample_count].imshow(overlay)
        axes[1][sample_count].set_title(f"Pred: {display_names[pred_label]}\n({conf
        axes[1][sample_count].axis("off")

        sample_count += 1
    if sample_count >= n_samples:
        break

plt.tight_layout()
fig.savefig(OUT_DIR / "12_gradcam.png", dpi=150)
plt.show()
```



Part 6 — TFLite Export & Quantization

Two quantization strategies:

1. **Post-Training Dynamic Range Quantization** — fast, ~4x size reduction
2. **Post-Training Full Integer Quantization (INT8)** — max compression, uses representative dataset for calibration

Note: QAT via `tensorflow-model-optimization` is incompatible with TF 2.21. Full INT8 PTQ with a representative calibration dataset achieves comparable results for binary classification tasks.

In [173…

```
# ── 6.1 Save full Keras model ──────────────────────────────────────
model.save(str(OUT_DIR / "onion_model_full.keras"))
full_size_mb = os.path.getsize(OUT_DIR / "onion_model_full.keras") / (1024 * 1024)
print(f"Full Keras model saved: {OUT_DIR / 'onion_model_full.keras'}")
print(f"Full model size: {full_size_mb:.2f} MB")
save_checkpoint("model_saved", {"full_size_mb": full_size_mb})
```

```
Full Keras model saved: /kaggle/working/caterpillar_output/onion_model_full.keras
Full model size: 12.12 MB
  [CHECKPOINT SAVED] model_saved
```

In [174…

```
# ── 6.2 Dynamic Range Quantization (float16) ──────────────────────
# Dynamic range gives ~3-4x compression with minimal accuracy loss
# Input/output stays float32 so no quantization mismatch issues

converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
# Float16 quantization: weights stored as float16, computed as float32
converter.target_spec.supported_types = [tf.float16]

tflite_model = converter.convert()

tflite_path = OUT_DIR / "onion_model.tflite"
with open(tflite_path, "wb") as f:
    f.write(tflite_model)

tflite_size_mb = os.path.getsize(tflite_path) / (1024 * 1024)
print(f"TFLite float16 model saved to: {tflite_path}")
```

```
print(f"TFLite model size: {tflite_size_mb:.2f} MB")
print(f"Compression ratio: {full_size_mb / tflite_size_mb:.1f}x")
```

```
INFO:tensorflow:Assets written to: /tmp/tmp8qyt8hsx/assets
```


```
INFO:tensorflow:Assets written to: /tmp/tmp8qyt8hsx/assets
```

```
Saved artifact at '/tmp/tmp8qyt8hsx'. The following endpoints are available:

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32,
name='keras_tensor_1242')
Output Type:
  TensorSpec(shape=(None, 6), dtype=tf.float32, name=None)
Captures:
  140291018354576: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018354000: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018343632: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018345744: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018346128: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018344400: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018354384: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018354192: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018356304: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018343440: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018342864: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018342672: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018352656: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018356496: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018349200: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018355152: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018357072: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018354768: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018356112: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018351696: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018347088: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018346704: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018351312: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018343248: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018355344: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140292492695824: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290993660048: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018344784: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018346320: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018354960: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140292678545296: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290993657360: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290993665424: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290993654096: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018351120: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018351888: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140292668906704: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140292668913808: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140292668904784: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140295031937104: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018349008: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290995598352: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140292668912464: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291420887888: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140292668904208: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018355920: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290995598160: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291474174800: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

```
140292668903824: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291420887696: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474173456: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474176720: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474172496: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474168656: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474170000: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474172304: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474176144: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474172112: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474173264: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474163280: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474168464: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474174224: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474170192: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474171536: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474163856: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474170576: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474170960: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474174416: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474175376: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474175760: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474175184: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474177296: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474168080: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474166544: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474172688: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474174032: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474176912: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291474169424: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952103888: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952110608: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952113488: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952103312: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952109264: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952106576: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952102736: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952106192: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952117520: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952116368: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952109072: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952112912: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952111760: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952109648: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952109840: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952115216: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952112144: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952117904: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952117712: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952117328: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952103120: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952107728: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952113680: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952115408: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952112720: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952118096: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

```
140290952112528: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952116176: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952113872: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952110992: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952114256: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952114064: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952115024: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952111568: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952113296: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952113104: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952108688: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952115792: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952110224: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952111184: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952107536: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952110416: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952115984: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952105232: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952108880: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952109456: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952105808: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952114832: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952105616: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952107920: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952106960: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952104272: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952111376: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952101968: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952106768: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952105424: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952106000: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952110032: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952103504: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952104656: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952104464: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952105040: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952106384: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952110800: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952112336: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952104080: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952102928: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952111952: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952102544: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952104848: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952107344: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952102160: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952102352: TensorSpec(shape=(), dtype=tf.resource, name=None)
140290952107152: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008478288: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008478672: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008478480: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008477520: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008477712: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008462928: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008477904: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008478096: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

```
140291008479056: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008478864: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008049424: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008048080: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008048272: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008047888: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008045200: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008041168: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008046544: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008037712: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008047120: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008036944: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008047696: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008049040: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008037904: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008037136: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008038288: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008047504: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008044048: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008038096: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008038864: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008037328: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008045584: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008038672: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008047312: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008039824: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008039248: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008039440: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008039632: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008037520: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008040784: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008040208: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008038480: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008041744: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008040976: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008042128: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008040400: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008039056: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008041936: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008042704: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008041360: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008040016: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008042512: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008040592: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008043664: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008043088: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008043280: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008043472: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008041552: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008044816: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008044240: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008042320: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008045968: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008045008: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008046352: TensorSpec(shape=(), dtype=tf.resource, name=None)
140291008044432: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

```
  140291008042896: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291008046160: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291008050000: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291008045392: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291008043856: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291008049808: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291008044624: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291008050960: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291008050384: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291008050576: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291008050768: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291008045776: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291008051920: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291008051344: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291008049616: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291008052688: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291008053072: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291008051728: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291008052496: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291008052880: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291008051536: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291008051152: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291008050192: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290949841744: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290949841552: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291008052304: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291008052112: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290949840976: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290949842128: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290949841936: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290949841168: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290949843280: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290949842704: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290949842320: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290949844048: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290949843472: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290949844432: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290949842896: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290949841360: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290949844240: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290949845008: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290949843664: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290949842512: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290949844816: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290949843088: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290949845968: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290949845584: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290949846544: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140291018345168: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290949846352: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290949846928: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140290949846736: TensorSpec(shape=(), dtype=tf.resource, name=None)
W0000 00:00:1775455545.754346      55 tf_tfl_flatbuffer_helpers.cc:365] Ignored outp
ut_format.
W0000 00:00:1775455545.754424      55 tf_tfl_flatbuffer_helpers.cc:368] Ignored drop
_control_dependency.
```

```
TFLite float16 model saved to: /kaggle/working/caterpillar_output/onion_model.tflite
TFLite model size: 5.72 MB
Compression ratio: 2.1x
```

In [175…

```python
# ── 6.3 Validate TFLite model accuracy ──────────────────────────
interpreter = tf.lite.Interpreter(model_path=str(tflite_path))
interpreter.allocate_tensors()
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()

print(f"Input:  shape={input_details[0]['shape']}, dtype={input_details[0]['dtype']
print(f"Output: shape={output_details[0]['shape']}, dtype={output_details[0]['dtype

tflite_correct = 0
tflite_total = 0

for imgs, lbls in test_pipeline:
    for i in range(len(imgs)):
        # Input is float32 [-1, 1] — no quantization needed
        img_input = np.expand_dims(imgs[i].numpy(), axis=0).astype(np.float32)

        interpreter.set_tensor(input_details[0]["index"], img_input)
        interpreter.invoke()
        output = interpreter.get_tensor(output_details[0]["index"])

        pred_label = np.argmax(output[0])
        true_label = np.argmax(lbls[i].numpy())

        if pred_label == true_label:
            tflite_correct += 1
        tflite_total += 1

tflite_acc = tflite_correct / tflite_total
print(f"\nTFLite float16 Test Accuracy: {tflite_acc:.4f} ({tflite_acc*100:.2f}%)")
print(f"Full model accuracy:          {test_acc:.4f} ({test_acc*100:.2f}%)")
print(f"Accuracy drop (TFLite):       {(test_acc - tflite_acc)*100:.2f}%")
print(f"\nModel size: {tflite_size_mb:.2f} MB (target: <8 MB) {'PASS' if tflite_siz

save_checkpoint("tflite_eval", {
    "tflite_acc": tflite_acc, "full_acc": test_acc,
    "tflite_size_mb": tflite_size_mb, "full_size_mb": full_size_mb,
})
```

```
Input:  shape=[  1 224 224   3], dtype=<class 'numpy.float32'>
Output: shape=[1 6], dtype=<class 'numpy.float32'>
/usr/local/lib/python3.12/dist-packages/tensorflow/lite/python/interpreter.py:457: U
serWarning:     Warning: tf.lite.Interpreter is deprecated and is scheduled for dele
tion in
    TF 2.20. Please use the LiteRT interpreter from the ai_edge_litert package.
    See the [migration guide](https://ai.google.dev/edge/litert/migration)
    for details.

  warnings.warn(_INTERPRETER_DELETION_WARNING)
```

```
TFLite float16 Test Accuracy: 0.9375 (93.75%)
Full model accuracy:          0.9375 (93.75%)
Accuracy drop (TFLite):       0.00%

Model size: 5.72 MB (target: <8 MB) PASS
  [CHECKPOINT SAVED] tflite_eval
```

Final Summary

In [176…

```python
# ── Final Summary ──────────────────────────────────────────────────
print("=" * 60)
print("ONION DISEASE & PEST MODEL — FINAL REPORT")
print("=" * 60)
print(f"""
Model:           MobileNetV3-Large (3-stage progressive fine-tuning)
Classes:         {', '.join(CLASS_NAMES)}
Num Classes:     {NUM_CLASSES}
Input:           {IMG_SIZE}x{IMG_SIZE}x3, normalized [-1,1]
Optimizer:       AdamW (weight_decay=0.01)
LR Schedule:     Cosine annealing with linear warmup
Augmentation:    RandomFlip/Rotation/Zoom/Brightness/Contrast
Regularization:  Dropout(0.3) + Class Weights
Quantization:    Full INT8 PTQ with representative dataset calibration

RESULTS:
  Full Model Accuracy:    {test_acc*100:.2f}%
  TFLite INT8 Accuracy:   {tflite_acc*100:.2f}%
  Full Model Size:        {full_size_mb:.2f} MB
  TFLite Model Size:      {tflite_size_mb:.2f} MB

SAVED TO /kaggle/working/:
  {OUT_DIR}/onion_model_full.keras
  {OUT_DIR}/onion_model.tflite
""")
print("Pipeline complete! Download your models from the Output tab.")
```

```
============================================================
ONION DISEASE & PEST MODEL — FINAL REPORT
============================================================

Model:           MobileNetV3-Large (3-stage progressive fine-tuning)
Classes:         Alternaria, Bulb_Blight, Caterpillar, Fusarium, Healthy, Virosis
Num Classes:     6
Input:           224x224x3, normalized [-1,1]
Optimizer:       AdamW (weight_decay=0.01)
LR Schedule:     Cosine annealing with linear warmup
Augmentation:    RandomFlip/Rotation/Zoom/Brightness/Contrast
Regularization:  Dropout(0.3) + Class Weights
Quantization:    Full INT8 PTQ with representative dataset calibration

RESULTS:
  Full Model Accuracy:    93.75%
  TFLite INT8 Accuracy:   93.75%
  Full Model Size:        12.12 MB
  TFLite Model Size:      5.72 MB

SAVED TO /kaggle/working/:
  /kaggle/working/caterpillar_output/onion_model_full.keras
  /kaggle/working/caterpillar_output/onion_model.tflite

Pipeline complete! Download your models from the Output tab.
```